

Neo4j × HackFront India

Mini-Hack: Problem Statements & Judging Framework

Date: 26 September 2026

Venue: DataPhi, Pune

Format: Workshop + 2-Hour Build Sprint + Team Demonstrations

Participants: Students + Working Professionals

Team Size: 2–4 participants

1. Mini-Hack Theme

Build an AI Agent That Remembers

Participants will build an AI-powered prototype using **Neo4j as a meaningful part of the memory and relationship layer**.

The objective is to demonstrate how persistent memory and connected information can make AI agents more **contextual, personalized and useful**.

Participants can choose **one of two problem statements**:

1. **Context-Aware Customer Support Agent**
2. **Personal Productivity & Decision Memory Agent**

Both challenges will use the **same judging framework**.

2. Build Sprint Scope

The build sprint will be limited to approximately **2 hours**.

Participants are therefore expected to build a **focused functional prototype**, rather than a production-ready system.

Participants should focus on:

- One clearly defined user journey
- One meaningful memory workflow
- A functional AI agent
- Meaningful Neo4j integration
- Demonstrable memory retrieval
- A clear before/after demonstration

Participants are NOT expected to build:

- Production-grade authentication
- Complete enterprise workflows
- Large-scale datasets
- Complex front-end applications
- Fully autonomous multi-agent systems
- Production-level security/scalability

The quality of the **core Agent Memory + Neo4j implementation** will be more important than the number of features.

3. Common Technical Requirements

Regardless of the problem statement selected, every team should demonstrate:

1. AI Agent

An AI-powered agent capable of interacting with a user and/or performing a defined task.

2. Persistent Memory

The agent should retain relevant information beyond a single interaction.

3. Neo4j Integration

Neo4j should meaningfully represent, store, connect and/or retrieve relevant information.

4. Contextual Retrieval

The agent should retrieve relevant information from previously stored memory or connected entities.

5. Memory-Driven Improvement

The team should demonstrate that the agent provides a more contextual, personalized or useful response because of its memory.

4. Problem Statement 1

Context-Aware Customer Support Agent

Challenge

Build an AI-powered **Customer Support Agent** that remembers previous customer interactions and uses relationships between the customer, product, issue and previous resolution to provide more contextual support.

The goal is to demonstrate how persistent memory can reduce repetitive conversations and improve the quality of customer support.

Example Scenario

A customer contacts support:

Customer: "My course access isn't working. I purchased the course last week and already contacted support about this."

The agent should capture the relevant information.

Later, the customer returns:

Customer: "It's still not working."

Instead of treating this as a completely new conversation, the agent should retrieve the relevant history and respond based on the previous interaction.

For example, it could identify:

Customer → Product → Previous Issue → Previous Support Interaction → Resolution/Status

and use this context in its response.

Expected Prototype

Within the 2-hour build sprint, teams should focus on one support journey.

A possible flow:

Step 1 — Customer Interaction

Customer reports an issue.

↓

Step 2 — Memory Creation

Relevant customer, product and issue information is stored in Neo4j.

↓

Step 3 — New Interaction

The customer returns with a follow-up question.

↓

Step 4 — Memory Retrieval

The agent retrieves relevant previous interactions and relationships.

↓

Step 5 — Contextual Response

The agent responds using the retrieved information.

Suggested Data Model

Teams may use a simple graph such as:

Customer → Order → Product

Customer → Support Ticket → Issue

Support Ticket → Resolution

Customer → Previous Interaction

Participants may extend this model where useful.

Possible Extensions

Teams with additional time may explore:

- Similar historical support issues
- Customer preferences
- Product-specific issue patterns
- Previous resolutions
- Escalation recommendations
- Support-agent handoff
- Identifying unresolved issues

These are optional and should **not be required** for judging.

5. Problem Statement 2

Personal Productivity & Decision Memory Agent

Challenge

Build an AI-powered **Personal Productivity & Decision Memory Agent** that remembers a user's preferences, tasks, decisions, commitments and previous outcomes.

The agent should use this accumulated context to provide more personalized recommendations and assist the user in making or revisiting decisions.

Example Scenario

During an initial interaction, a user tells the agent:

"I prefer meetings in the morning and usually need two days to prepare for important client presentations."

The user also makes a decision:

"I'll prepare the presentation on Monday because the client meeting is on Wednesday."

The agent stores this information.

Later, the user asks:

"Help me plan my week."

The agent should use the user's previous preferences, commitments and decisions to create a more relevant plan.

Expected Prototype

Within the 2-hour build sprint, teams should focus on one productivity/decision workflow.

A possible flow:

Step 1 — User Interaction

User provides preferences, tasks or makes a decision.

↓

Step 2 — Memory Creation

Relevant information is stored in Neo4j.



Step 3 — New Interaction

User asks for a recommendation, schedule or decision support.



Step 4 — Memory Retrieval

The agent retrieves relevant preferences, tasks, decisions or outcomes.



Step 5 — Personalized Recommendation

The agent provides a recommendation using the retrieved context.

Suggested Data Model

Teams may use a simple graph such as:

User → Preference

User → Task

User → Decision → Reason

Decision → Outcome

User → Commitment/Event

Participants may extend the model where useful.

Possible Extensions

Teams with additional time may explore:

- Decision history
- Decision → outcome tracking
- Learning from previous outcomes
- Conflicting preferences

- Priority-based planning
- Recurring tasks
- Personalized recommendations
- "Why did I make this decision?" queries

These are optional.

6. Recommended Starter Resources

To make both challenges achievable within approximately two hours, participants should ideally receive a common starter environment.

Suggested starter kit

- Neo4j database/environment
- Basic Neo4j schema/example
- Sample data
- Basic LLM/API integration
- Starter agent code
- Example Neo4j queries
- Challenge documentation
- Submission template

Participants can then spend the majority of the sprint on **problem-solving and implementation**, rather than environment setup.

7. Common Judging Criteria

Both problem statements will be evaluated using the same framework.

Criteria	Weightage
Meaningful Use of Neo4j & Graph Thinking	25
Agent Memory & Contextual Retrieval	25
Problem-Solution Fit	20
Working Implementation	15
Demo & User Experience	10
Innovation / Creativity	5
Total	100

7.1 Meaningful Use of Neo4j & Graph Thinking — 25 Points

Judges will evaluate:

- How meaningfully Neo4j is used
- Quality of entity and relationship modelling
- Whether relationships contribute to the solution
- How graph-based retrieval is used
- Whether Neo4j is appropriate for the problem

Key question:

Is Neo4j genuinely enabling the solution, or is it simply being used as a database?

7.2 Agent Memory & Contextual Retrieval — 25 Points

Judges will evaluate:

- What information the agent remembers
- Whether memory persists
- Whether relevant memory is retrieved
- Accuracy/relevance of retrieved context
- Whether retrieved memory changes the agent's response or action

Key question:

Does the agent become meaningfully better because it remembers?

7.3 Problem-Solution Fit — 20 Points

Judges will evaluate:

- Clarity of the problem
 - Relevance of the proposed solution
 - Usefulness to the intended user
 - Whether Agent Memory meaningfully contributes to solving the problem
-

7.4 Working Implementation — 15 Points

Judges will evaluate:

- Functional prototype
- End-to-end working flow
- Reliability of the demonstrated functionality
- Completion of the core challenge requirements

Given the limited build time, **a small working prototype is preferable to a large incomplete system.**

7.5 Demo & User Experience — 10 Points

Judges will evaluate:

- Clarity of demonstration
 - Ease of understanding
 - User experience
 - Ability to clearly communicate the value of memory
 - Quality of live demonstration
-

7.6 Innovation / Creativity — 5 Points

Judges will evaluate:

- Creative application of Agent Memory
 - Interesting use of graph relationships
 - Novel approach to the selected problem
-

8. Mandatory Demonstration

Every shortlisted team must **show the memory working.**

The demonstration should follow this sequence:

1. Teach the Agent

Provide the agent with information about the customer/user, issue, preference, task or decision.

↓

2. Store the Memory

Show how the information is represented/stored using Neo4j.

↓

3. Start a New Interaction

Ask a new question where the previous information is relevant.

↓

4. Retrieve Context

Show that the agent retrieves the appropriate information from Neo4j/memory.



5. Demonstrate the Improvement

Show how the response is more contextual, personalized or useful because of the retrieved memory.

9. Final Demonstration Format

Each shortlisted team will receive:

3 Minutes — Live Demonstration

1 Minute — Jury Q&A

Recommended Demo Flow

0:00-0:30 — Problem

Explain the problem and target user.

0:30-1:30 — Initial Interaction

Show the information being provided to the agent and stored as memory.

1:30-2:30 — Memory in Action

Start a subsequent interaction and show how the agent retrieves and uses previous information.

2:30-3:00 — Neo4j

Show the relevant graph and explain how Neo4j enables the memory/contextual retrieval.

Jury Q&A — 1 Minute

Possible questions:

- Why did you choose Neo4j?
 - What information are you storing as memory?
 - How do you decide which memory to retrieve?
 - How does the graph structure improve the agent?
 - What happens if irrelevant memory is retrieved?
 - What would you build next with more development time?
-

10. Submission Requirements

Each team should submit:

1. Working Prototype

A functional demonstration of the solution.

2. Source Code

GitHub repository or equivalent.

3. Architecture Diagram

One-slide architecture showing:

User → Agent → Memory/Retrieval → Neo4j → Response

along with other major components.

4. Solution Summary

A short description covering:

- Problem
- Target user
- Solution
- What the agent remembers
- How Neo4j is used
- How memory improves the experience

11. Success Criteria for the Mini-Hack

The challenge is successful if participants can demonstrate, within the available build time:

A working AI agent + persistent memory + meaningful Neo4j graph usage + contextual retrieval + a clear improvement in the agent's response.

The event should prioritize **depth of one working use case over breadth of features.**

A simple, reliable prototype that clearly demonstrates Agent Memory and Neo4j should be considered stronger than a larger but incomplete application.